

# A System-Oriented Framework for SPAD Image Reconstruction

Junwen Gu

April 20, 2026

## Abstract

Single-Photon Avalanche Diodes (SPADs) enable imaging in extremely low-light regimes, but the resulting measurements are dominated by severe photon sparsity and signal-dependent shot noise. In practice, SPAD reconstruction is not only a modeling problem but also a systems problem: compressed bitstream decoding, CPU-side preprocessing, and data movement can become the dominant bottlenecks long before modern GPUs are saturated. This report presents a system-oriented SPAD reconstruction pipeline for high-resolution RGB recovery from burst measurements. The proposed framework combines a two-stage cascaded SE-ResUNet architecture with a profiling-driven training and inference stack, including robust preprocessing, in-memory staging, multiprocessing, mixed precision, and asynchronous writeback. The goal of this report is to emphasize the engineering decisions and design trade-offs that made large-scale SPAD reconstruction practical and stable.

## 1 Introduction

Single-photon imaging is attractive because it enables visual recovery under extremely limited light, but the measurements are highly sparse and noisy. Unlike standard denoising pipelines designed around approximately Gaussian noise, SPAD observations are governed by photon counting statistics and compressed binary representations. This makes reconstruction challenging at both the algorithmic and systems levels.

A second practical issue is that large-scale SPAD training often becomes input-bound rather than compute-bound. In preliminary profiling, the main bottlenecks were not forward or backward passes on the GPU, but rather com-

pressed chunk loading, bit unpacking, CPU-side preprocessing, and data transfer overhead. As a result, naive end-to-end training left substantial GPU capacity underutilized even on high-end hardware.

This work addresses both sides of the problem. On the modeling side, a two-stage coarse-to-fine reconstruction cascade is used to improve stability and reduce physically unsupported texture generation. On the systems side, the full pipeline is organized around robust input normalization, in-memory staging, multiprocessing, mixed precision, and asynchronous output handling. The resulting framework is intended to be portable, reproducible, and practical for large SPAD reconstruction workloads.

**Contributions.** The main contributions of this work are:

- A two-stage SE-ResUNet reconstruction pipeline for coarse structure recovery followed by residual refinement.
- A robust preprocessing stack that converts heterogeneous raw SPAD chunks into consistent 48-channel tensors.
- A profiling-driven systems pipeline that improves throughput through in-memory staging, multiprocessing, AMP, and asynchronous writeback.
- A modular inference workflow suitable for large-scale evaluation on held-out SPAD scenes.

## 2 Problem Setting and Data Preparation

### 2.1 SPAD Reconstruction Setting

The goal is to recover a high-quality RGB image from burst measurements produced by a SPAD sensor. Each

raw sample consists of 1024 temporal bitplanes, while the desired reconstruction corresponds to the final image associated with the burst. Because the raw data are compressed and sparse, successful reconstruction depends on both feature extraction and efficient data handling.

## 2.2 Input Normalization

In practice, raw chunks may appear in slightly different formats, such as  $(1024, H, W)$  or  $(1024, H, W, 3)$ . To avoid brittle dataset-specific assumptions, the loader first normalizes all inputs into a consistent representation. The downstream network always receives tensors of the form

$$(B, 48, 800, 800),$$

where  $48 = 16 \times 3$  corresponds to 16 temporal proxy slices with 3 channels each.

## 2.3 Preprocessing Pipeline

The preprocessing pipeline is designed to be simple, robust, and compatible with both training and inference:

1. **Temporal slicing:** retain the final 192 time bins, which were found empirically to contain the most informative late-arriving photon events.
2. **Bit unpacking:** decode compressed bitstreams into expanded spatial tensors.
3. **Temporal aggregation:** divide the retained time bins into 16 groups and average within each group to form temporal proxy images.
4. **Normalization and enhancement:** apply percentile normalization, logarithmic compression, and light Gaussian smoothing.
5. **Tensor reshaping:** flatten the resulting  $16 \times 3$  representation into a fixed 48-channel input.

This preprocessing choice is intentionally conservative. It reduces variance and improves early optimization stability, while still preserving temporal diversity across channel groups.

# 3 Method

## 3.1 Two-Stage Coarse-to-Fine Reconstruction

A single expressive network can become unstable in extremely sparse photon regimes, especially when the model begins generating visually plausible but physically unsupported textures. To improve stability, a cascaded architecture is used.

**Stage 1: Coarse Reconstruction.** The first stage reconstructs global layout, object boundaries, and illumination structure from the 48-channel SPAD feature tensor. It is implemented as an SE-ResUNet with residual convolution blocks and channel-wise recalibration.

**Stage 2: Residual Refinement.** The second stage takes the coarse RGB prediction and learns a residual correction, rather than predicting the full image from scratch. This focuses the refinement network on structured error correction and texture recovery.

## 3.2 Residual Formulation

Let  $I_{\text{coarse}}$  denote the Stage 1 output and  $I_{\text{GT}}$  the ground truth image. Stage 2 predicts a residual

$$R = I_{\text{GT}} - I_{\text{coarse}},$$

and the final reconstruction is

$$I_{\text{out}} = I_{\text{coarse}} + R.$$

This formulation reduces the burden on the second stage, since low-frequency content and scene geometry are already present in the coarse prediction.

## 3.3 SE-ResUNet Backbone

Each stage uses a U-Net-like encoder-decoder with residual convolutional blocks. In Stage 1, Squeeze-and-Excitation blocks help reweight channels adaptively, which is useful because different temporal slices contribute unevenly depending on scene content and photon density. This is especially relevant in SPAD bursts, where signal quality varies strongly across temporal bins.

## 4 Training and Optimization

### 4.1 Loss Design

A composite loss is used to balance pixel fidelity and structural quality:

- $\ell_1$  **loss** for stable pixel-level supervision under heavy noise,
- **SSIM loss** for local structural consistency,
- **Perceptual features** explored selectively during experimentation, with care taken to avoid over-emphasizing hallucinated texture.

### 4.2 Optimization Strategy

Training stability and throughput are improved by several standard but important choices:

- **Optimizer:** AdamW,
- **Learning rate schedule:** cosine annealing,
- **Mixed precision:** AMP to reduce VRAM pressure and increase throughput,
- **EMA weights:** Exponential Moving Average for smoother evaluation checkpoints.

In addition, selected high-detail scenes were sampled more frequently during later-stage refinement experiments in order to reduce oversmoothing and preserve textured structure.

## 5 System Engineering

### 5.1 Why the Pipeline Was Input-Bound

Profiling showed that raw training throughput was limited not by the model itself, but by upstream processing:

- compressed chunk loading,
- CPU-bound bit unpacking,
- repeated decompression,
- host-to-device transfer overhead.

This meant that simply scaling up GPU hardware did not automatically improve training speed.

Table 1: Quantitative performance comparison across reconstruction stages.

| Model Stage           | PSNR (dB)    | SSIM          | $\ell_1$ Loss |
|-----------------------|--------------|---------------|---------------|
| Stage 1 (Base)        | 23.77        | 0.6867        | 0.0487        |
| Stage 2 (EMA)         | 30.56        | 0.8734        | 0.0182        |
| Stage 2 (EMA2 Hybrid) | <b>31.45</b> | <b>0.8807</b> | <b>0.0165</b> |

### 5.2 In-Memory Staging

To reduce repeated storage overhead, intermediate data and decompressed artifacts were staged in an in-memory filesystem when available. In I/O-bound cases, this significantly reduced waiting time relative to repeatedly pulling from slower storage.

### 5.3 Multiprocessing and Parallel Data Handling

The dataset pipeline was restructured to support multiprocessing safely and consistently. This improved the overlap between CPU preprocessing and GPU training and reduced stalls in the training loop. In profiling runs, these changes resulted in sustained high GPU utilization during long training segments.

### 5.4 Asynchronous Writeback

Checkpoint and output writing were offloaded asynchronously to reduce blocking behavior. This was particularly helpful for larger runs where frequent synchronous writes introduced noticeable variance in step time.

## 6 Results

### 6.1 Quantitative Results

All results are evaluated on held-out scenes from the SPAD reconstruction dataset. Table 1 summarizes representative improvements across training stages.

The cascaded design improves reconstruction quality substantially over the initial coarse stage. In particular, residual refinement improves both PSNR and SSIM while reducing pixel-level error. Qualitatively, the second stage

is most helpful in suppressing structured artifacts and recovering moderate texture without destabilizing the global image layout.

## 6.2 Qualitative Observations

Several qualitative patterns were consistent across experiments:

- **Very dark scenes:** the first stage recovers the global structure, while the second stage improves boundaries and suppresses visible artifacts.
- **Textured regions:** the cascaded model reduces over-smoothing compared with simpler single-stage baselines, although fine texture remains challenging in the lowest-photon regime.
- **Stability:** EMA checkpoints generally produced cleaner and more consistent reconstructions than raw checkpoints.

## 7 Reproducibility and Deliverables

The project was organized as a portable PyTorch workflow with modular components:

- `model_arch.py`: network definitions for Stage 1 and Stage 2,
- `utils.py`: decoding and preprocessing utilities,
- `inference.py`: recursive scene-level reconstruction for evaluation,
- pretrained checkpoints for coarse reconstruction and refinement.

The inference pipeline is designed to run in standard PyTorch environments with minimal dependencies beyond NumPy, OpenCV, and tqdm.

## 8 Limitations and Future Work

Although the current system is practical and stable, several limitations remain.

First, the preprocessing pipeline deliberately compresses the temporal dimension into a compact fixed representation. This improves robustness and efficiency, but may discard

fine-grained temporal information that could help preserve subtle details.

Second, the second stage is fundamentally limited by the information preserved in the coarse prediction. If high-frequency details are removed early, refinement can only partially recover them.

Future work will therefore focus on:

- **Temporal modeling:** attention or sequence-based methods across temporal bins,
- **Physics-aware objectives:** losses more tightly aligned with Poisson statistics,
- **Lower-level optimization:** reducing unpacking overhead through vectorization or compiled extensions,
- **Dynamic-scene robustness:** improving consistency under motion and burst misalignment.

## 9 Conclusion

This report presented a system-oriented SPAD reconstruction framework that combines a two-stage cascaded SE-ResUNet with a practical high-throughput training and inference pipeline. The main lesson is that SPAD reconstruction quality depends not only on model design, but also on robust preprocessing and efficient data movement. By treating the problem as both a reconstruction task and a systems problem, the resulting framework achieves stable training, portable inference, and strong reconstruction performance under severe photon sparsity.

## References